

ASSIGNMENT – 2b

Multi-Client Student Record Retrieval Using TCP and UDP Sockets

AIM

To develop a client-server application in Python that lets students retrieve their academic information from a university server. The server reads student records from a CSV file (students.csv) containing Roll Number, Name, Department, Semester, and CGPA, and serves them to clients over the network.

Two implementations are built: a TCP version (1a) where the server accepts multiple clients concurrently using threads and returns the complete academic profile for the requested Roll Number, and a UDP version (1b) where the client checks whether a Roll Number exists and receives the student's name as a datagram. In both versions the server returns "Student Record Not Found." for unknown Roll Numbers. The two implementations are then compared (1c) on connection, reliability, ordering, error recovery, and communication.

DATASET – students.csv

Sample records used by both servers

```
Roll Number,Name,Department,Semester,CGPA
3122245001312,Srikrishna O S,CSE,4,8.92
3122245001001,Arjun Kumar,CSE,4,8.45
3122245001002,Priya Sharma,ECE,4,7.89
3122245001003,Rahul Menon,IT,6,9.10
3122245001004,Sneha Iyer,EEE,6,8.30
3122245001005,Vikram Singh,MECH,2,7.55
3122245001006,Divya Nair,CSE,8,9.01
3122245001007,Karthik Raja,CIVIL,4,6.78
```

1a. TCP – SERVER CODE

```
import csv
import socket
import threading

# student record server, reads students.csv and serves records over tcp

def load_students():
    records = {}
    with open("students.csv", encoding="utf-8") as f:
```

```
        for row in csv.DictReader(f):
            records[row["Roll Number"].strip()] = row
    return records

records = load_students()
print(f"Loaded {len(records)} student records from students.csv")

print_lock = threading.Lock()

def log(msg):
    # lock so two threads never write into the same line
    with print_lock:
        print(msg)

def handle_client(conn, addr):
    log(f"Client Connected: {addr}")
    try:
        roll = conn.recv(1024).decode().strip()
        log(f"[Client {addr[1]}] Requested Roll Number: {roll}")
        if roll.lower() == "exit":
            return
        if roll in records:
            r = records[roll]
            reply = (
                f"Roll Number: {r['Roll Number']}\n"
                f"Name: {r['Name']}\n"
                f"Department: {r['Department']}\n"
                f"Semester: {r['Semester']}\n"
                f"CGPA: {r['CGPA']}"
            )
            log(f"[Client {addr[1]}] Record found, sending details")
        else:
            reply = "Student Record Not Found."
            log(f"[Client {addr[1]}] Record not found")
        conn.sendall(reply.encode())
    except ConnectionResetError:
        pass
    finally:
        log(f"Client {addr} disconnected.")
        conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 5002))
server.listen(5)
```

```
print("Student Record Server Started...")
print("Server listening on 127.0.0.1:5002")

while True:
    conn, addr = server.accept()
    # one thread per client so everyone is served at the same time
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

1a. TCP – CLIENT CODE

```
import socket

# student client: asks for one roll number and prints what the server sends back

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5002))

roll = input("Enter Roll Number: ")
client.sendall(roll.encode())

reply = client.recv(1024).decode()
print("Server Response:")
print(reply)

client.close()
print("Disconnected from Server.")
```

1a. TCP – SERVER OUTPUT

```
Loaded 8 student records from students.csv
Student Record Server Started...
Server listening on 127.0.0.1:5002
Client Connected: ('127.0.0.1', 51702)
Client Connected: ('127.0.0.1', 51703)
[Client 51702] Requested Roll Number: 3122245001002
[Client 51702] Record found, sending details
Client ('127.0.0.1', 51702) disconnected.
[Client 51703] Requested Roll Number: 9999999999
[Client 51703] Record not found
Client ('127.0.0.1', 51703) disconnected.
Client Connected: ('127.0.0.1', 51704)
[Client 51704] Requested Roll Number: exit
Client ('127.0.0.1', 51704) disconnected.
```

1a. TCP – CLIENT 1 OUTPUT (RECORD FOUND)

```
Enter Roll Number: 3122245001002
Server Response:
Roll Number: 3122245001002
Name: Priya Sharma
Department: ECE
Semester: 4
CGPA: 7.89
Disconnected from Server.
```

1a. TCP – CLIENT 2 OUTPUT (NOT FOUND)

```
Enter Roll Number: 9999999999
Server Response:
Student Record Not Found.
Disconnected from Server.
```

1b. UDP – SERVER CODE

```
import csv
import socket

# udp student record server, checks roll number and sends back the name

def load_students():
    records = {}
    with open("students.csv", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            records[row["Roll Number"].strip()] = row
    return records

records = load_students()
print(f"Loaded {len(records)} student records from students.csv")

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 5003))
print("UDP Student Record Server Started...")
print("Server listening on 127.0.0.1:5003")

while True:
    data, addr = server.recvfrom(1024)
    roll = data.decode().strip()
```

```
print(f"[Client {addr[1]}] Requested Roll Number: {roll}")
if roll.lower() == "exit":
    # connectionless, nothing to close
    continue
if roll in records:
    reply = f"Roll Number: {roll}\nName: {records[roll]['Name']}"
    print(f"[Client {addr[1]}] Record found, sending name")
else:
    reply = "Student Record Not Found."
    print(f"[Client {addr[1]}] Record not found")
server.sendto(reply.encode(), addr)
```

1b. UDP – CLIENT CODE

```
import socket

# udp student client: sends roll number and prints the server's reply

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

roll = input("Enter Roll Number: ")
client.sendto(roll.encode(), ("127.0.0.1", 5003))

reply, addr = client.recvfrom(1024)
print("Server Response:")
print(reply.decode())

client.close()
```

1b. UDP – SERVER OUTPUT

```
Loaded 8 student records from students.csv
UDP Student Record Server Started...
Server listening on 127.0.0.1:5003
[Client 50128] Requested Roll Number: 3122245001006
[Client 50128] Record found, sending name
[Client 50129] Requested Roll Number: 8888888888
[Client 50129] Record not found
[Client 50130] Requested Roll Number: exit
```

1b. UDP – CLIENT 1 OUTPUT (RECORD FOUND)

Enter Roll Number: 3122245001006
Server Response:
Roll Number: 3122245001006
Name: Divya Nair

1b. UDP – CLIENT 2 OUTPUT (NOT FOUND)

Enter Roll Number: 8888888888
Server Response:
Student Record Not Found.

1c. TCP vs UDP COMPARISON

Parameter	TCP	UDP
Connection	Connection-oriented: three-way handshake before data transfer and explicit close	Connectionless: datagrams are sent directly with no setup or teardown
Reliability	Reliable: guaranteed delivery with acknowledgments and retransmission of lost segments	Unreliable: best-effort delivery, datagrams may be lost without detection
Ordering	Ordered: segments are sequenced and delivered in the same order they were sent	Unordered: datagrams may arrive out of order
Error Recovery	Recovers from errors: ACKs, retransmission, and flow control	No recovery: only checksum-based error detection, damaged packets are dropped
Communication	Stream-oriented with higher overhead (larger headers, connection state)	Message-oriented with lower overhead (small 8-byte header), faster and simpler

TASKS PERFORMED

Task	Details	Result
Load student records	Read students.csv with csv.DictReader into a dictionary keyed by Roll Number	8 student records loaded by both servers
Create TCP server	Bind to 127.0.0.1:5002; spawn one thread per accepted client	Two clients connected and were served at the same time
TCP record found	Client sent 3122245001002; server searched the CSV and sent the full profile	Client received Roll Number, Name, Department, Semester, CGPA
TCP record not found	Client sent 9999999999; no matching record in the CSV	Server returned "Student Record Not Found."

Task	Details	Result
Create UDP server	Bind UDP socket to 127.0.0.1:5003; reply with sendto for every datagram	UDP Student Record Server Started; sender IP/port shown per request
UDP record found	Client sent 3122245001006; server confirmed the record and sent the name	Client received "Name: Divya Nair" as the datagram reply
UDP record not found	Client sent 8888888888; no matching record in the CSV	Server returned "Student Record Not Found."
Graceful termination	Client closes its connection after the reply; other clients unaffected	Server kept running and logged both disconnections cleanly

LEARNING OUTCOMES

- I read a CSV file with csv.DictReader and built a lookup dictionary keyed by Roll Number.
- I implemented a threaded TCP server that serves multiple student clients concurrently.
- I implemented a connectionless UDP server and clients using sendto() and recvfrom().
- I handled both found and not-found cases with the exact message "Student Record Not Found."
- I compared TCP and UDP on connection, reliability, ordering, error recovery, and communication.
- I observed that a print lock is needed when multiple threads log to the same console.